

CS-232 Lab Report 2

DATABASE MANAGEMENT SYSTEMS

Hamza Arif

Student ID: 2024206

BSc Artificial Intelligence

February 22, 2026

Task 1 Database Design and Table Creation

1.1 1. Departments Table

```
1 CREATE TABLE departments (  
2     dept_id SERIAL PRIMARY KEY,  
3     dept_name VARCHAR(50) UNIQUE NOT NULL,  
4     student_count INT  
5 );  
6  
7 INSERT INTO departments (dept_name, student_count)  
8 VALUES ('AI',50), ('ME',150), ('CS',50);
```

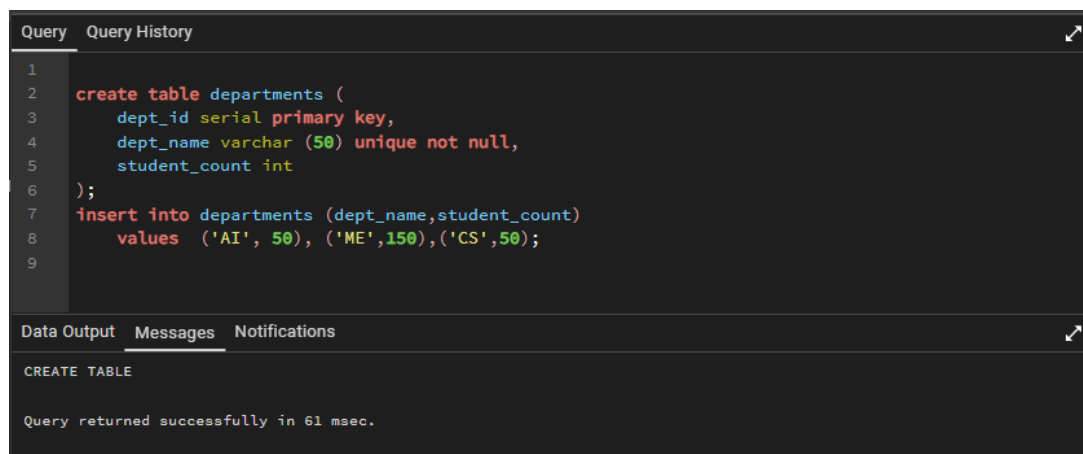


Figure 1: Departments Table Creation

1.2 2. Students Table

```
1 CREATE TABLE students (  
2     student_id SERIAL PRIMARY KEY,  
3     full_name VARCHAR(100) NOT NULL,  
4     email VARCHAR(100) UNIQUE,  
5     is_active BOOLEAN DEFAULT TRUE,  
6     dob DATE,  
7     admission_time TIMESTAMP,  
8     session_duration INTERVAL,  
9     skills TEXT,  
10    profile JSONB,  
11    cgpa NUMERIC(3,2) CHECK (cgpa BETWEEN 0 AND 4),  
12    dept_id INTEGER REFERENCES departments(dept_id)  
13 );
```

```

Query  Query History
3  create table students (
4      student_id serial,
5      full_name varchar(100) not null,
6      email varchar(100) unique,
7      is_active boolean default true,
8      dob date,
9      admission_time timestamp,
10     session_duration interval,
11     skills text,
12     profile jsonb,
13     cgpa NUMERIC(3,2) check (cgpa between 0 and 4),
14     dept_id integer,
15     constraint fk_students foreign key (dept_id) references departments(dept_id),
16     primary key (student_id)
17 );
18

Data Output  Messages  Notifications
CREATE TABLE
Query returned successfully in 92 msec.

```

Figure 2: Students Table Creation

1.3 Constraint Demonstrations

```

1 insert into students (full_name,email,is_active,dob,
   admission_time,session_duration,skills,profile,cgpa,dept_id)
2 values ('DUPLICATE_TEST_STUDENT','hamza@gmail.com',false,'
   29-06-2006','2023-09-01 09:30:00','4years','Python','{
   hobbies":["coding","sleepin"],address":{"city":"topi","
   country":"pakistan"}}',3.89,1),
3     ('CHECK_VIOLATION_TEST_STUDENT','alishah@gmail.
   com',false,'29-06-2006','2023-09-01 09:30:00',
   '4years','Python','{"hobbies":["coding","
   sleepin"],address":{"city":"topi","
   country":"pakistan"}}',5.0,1);
4 select * from students where is_active = false;

```

```

Data Output  Messages  Notifications
ERROR: duplicate key value violates unique constraint "students_email_key"
Key (email)=(hamza@gmail.com) already exists.

SQL state: 23505
Detail: Key (email)=(hamza@gmail.com) already exists.

Total rows:  Query complete 00:00:00.066

```

Figure 3: Unique Violation

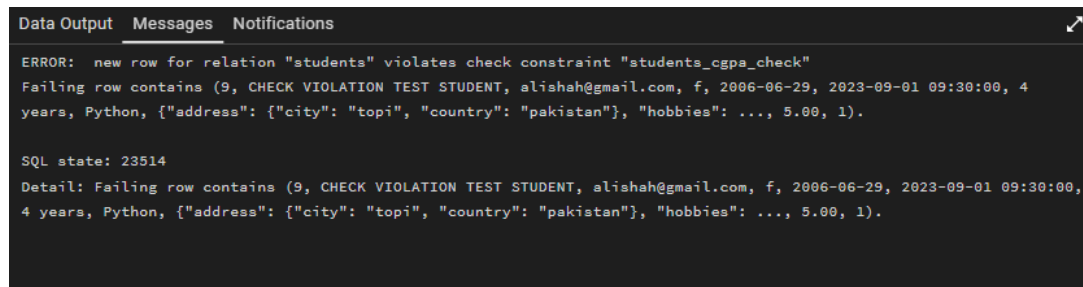


Figure 4: Check Violation

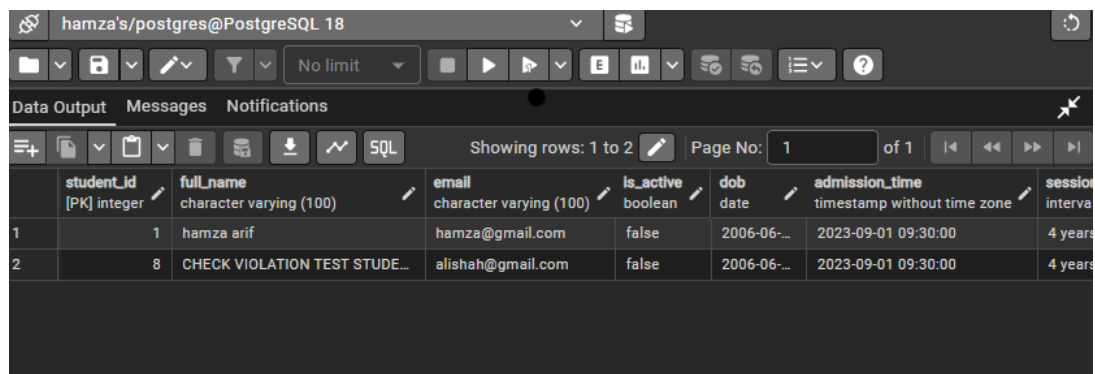


Figure 5: BOOLEAN filtering

1.4 Calculating age using DATE subtraction

```
1 select full_name, student_id, cgpa, (current_date - dob)/365 as
   age from students;
```

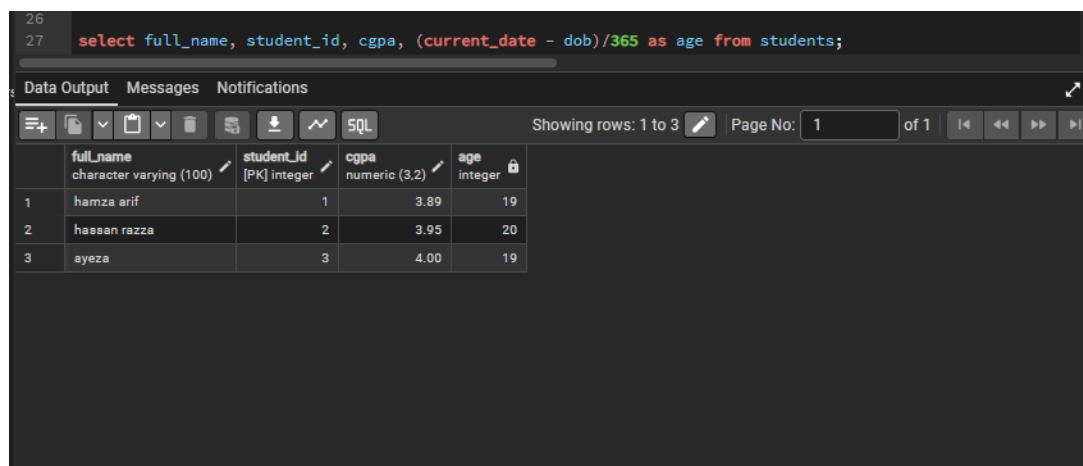
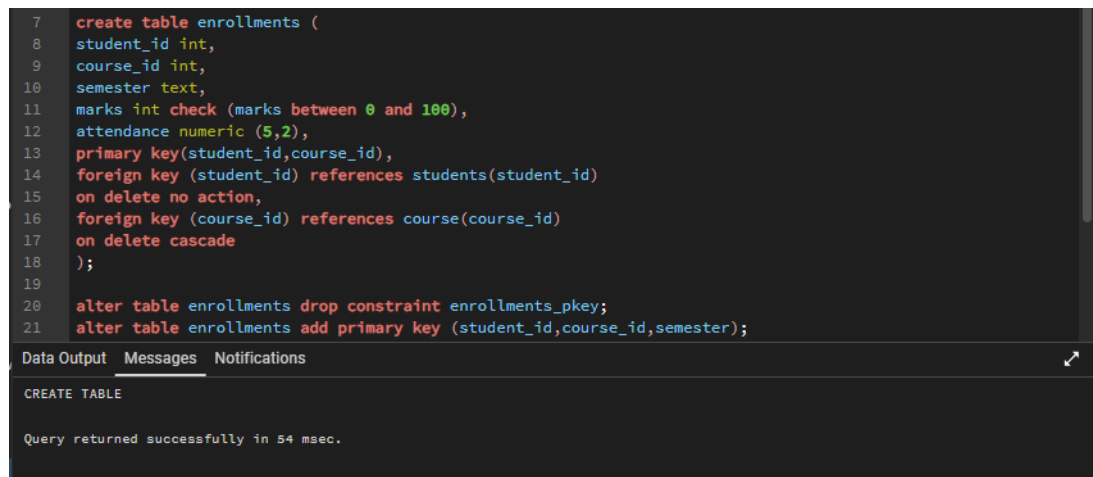


Figure 6: Age calculation

Task 2 Course Registration Policy

2.1 Enrollments Table with Composite Key

```
1 create table course(  
2 course_id serial unique,  
3 course_name varchar (30)  
4 );  
5  
6 create table enrollments (  
7 student_id int,  
8 course_id int,  
9 semester text,  
10 marks int check (marks between 0 and 100),  
11 attendance numeric (5,2),  
12 primary key(student_id,course_id),  
13 foreign key (student_id) references students(student_id)  
14 on delete no action,  
15 foreign key (course_id) references course(course_id)  
16 on delete cascade  
17 );
```



```
7 create table enrollments (  
8 student_id int,  
9 course_id int,  
10 semester text,  
11 marks int check (marks between 0 and 100),  
12 attendance numeric (5,2),  
13 primary key(student_id,course_id),  
14 foreign key (student_id) references students(student_id)  
15 on delete no action,  
16 foreign key (course_id) references course(course_id)  
17 on delete cascade  
18 );  
19  
20 alter table enrollments drop constraint enrollments_pkey;  
21 alter table enrollments add primary key (student_id,course_id,semester);
```

Data Output Messages Notifications

CREATE TABLE

Query returned successfully in 54 msec.

Figure 7: Enrollment table Creation

2.2 Altering Constraints

```
1 alter table enrollments drop constraint enrollments_pkey;  
2 alter table enrollments add primary key (student_id,course_id,  
3 semester);  
4 alter table enrollments add constraint semester_notnull check (  
5 semester is not null);  
6 alter table enrollments drop constraint semester_notnull;  
7 alter table enrollments add constraint atend_check check (  
8 attendance between 70 and 100) not valid;  
9 alter table enrollments validate constraint atend_check;  
10 select * from information_schema.table_constraints where  
11 table_name = 'enrollments';
```

```

20 alter table enrollments drop constraint enrollments_pkey;
21 alter table enrollments add primary key (student_id,course_id,semester)

```

Data Output Messages Notifications

ALTER TABLE

Query returned successfully in 75 msec.

Figure 8: Primary key addition

```

19
20 alter table enrollments drop constraint enrollments_pkey;
21 alter table enrollments add primary key (student_id,course_id,semester);
22 alter table enrollments add constraint semester_notnull check (semester is not null);
23 alter table enrollments drop constraint semester_notnull;
24 alter table enrollments add constraint atend_check check (attendance between 70 and 100) not valid;
25 alter table enrollments validate constraint atend_check;
26 select * from information_schema.table_constraints where table_name = 'enrollments';

```

Data Output Messages Notifications

Showing rows: 1 to 9 Page No: 1 of 1

	constraint_catalog name	constraint_schema name	constraint_name	table_catalog name	table_schema name	table_name	constraint_type	is_deferrable
1	hamza's	public	enrollments_marks_check	hamza's	public	enrollments	CHECK	NO
2	hamza's	public	enrollments_student_id_not_n...	hamza's	public	enrollments	CHECK	NO
3	hamza's	public	enrollments_course_id_not_n...	hamza's	public	enrollments	CHECK	NO
4	hamza's	public	enrollments_student_id_fkey	hamza's	public	enrollments	FOREIGN KEY	NO
5	hamza's	public	enrollments_course_id_fkey	hamza's	public	enrollments	FOREIGN KEY	NO
6	hamza's	public	enrollments_semester_not_null	hamza's	public	enrollments	CHECK	NO
7	hamza's	public	enrollments_pkey	hamza's	public	enrollments	PRIMARY KEY	NO
8	hamza's	public	semester_notnull	hamza's	public	enrollments	CHECK	NO
9	hamza's	public	atend_check	hamza's	public	enrollments	CHECK	NO

Figure 9: Alter table queries

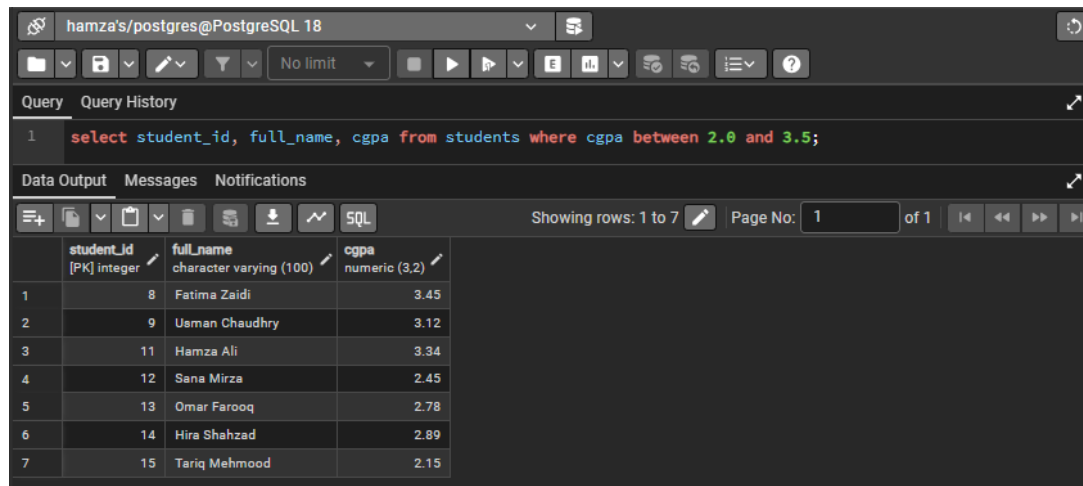
Task 3 Data Retrieval and Manipulation

3.1 Filtering Operations

```

1 select student_id, full_name, cgpa from students where cgpa
   between 2.0 and 3.5;
2
3 select student_id, full_name, cgpa from students where full_name
   like 'A%';
4
5 select student_id, full_name, cgpa from students where cgpa > 3.0
   and is_active = true;
6
7 select * from students where dept_id in (3,4);
8
9 select * from students where profile is null; --none have profile
   null

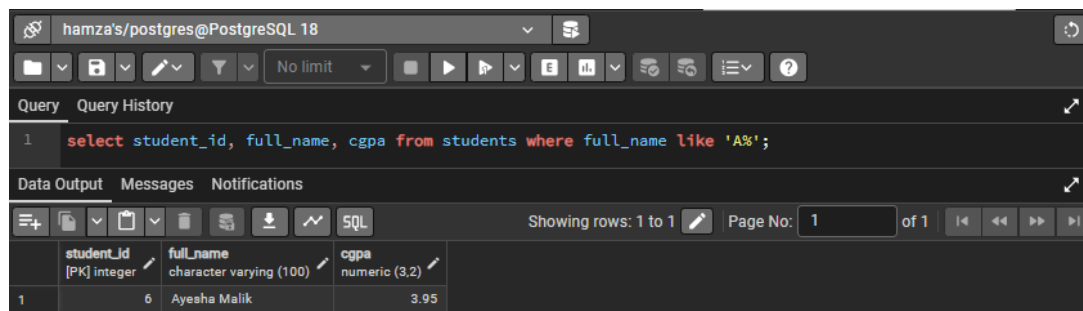
```



The screenshot shows a PostgreSQL query window with the following query: `select student_id, full_name, cgpa from students where cgpa between 2.0 and 3.5;`. The results are displayed in a table with 7 rows.

	student_id [PK] integer	full_name character varying (100)	cgpa numeric (3,2)
1	8	Fatima Zaidi	3.45
2	9	Usman Chaudhry	3.12
3	11	Hamza Ali	3.34
4	12	Sana Mirza	2.45
5	13	Omar Farooq	2.78
6	14	Hira Shahzad	2.89
7	15	Tariq Mehmood	2.15

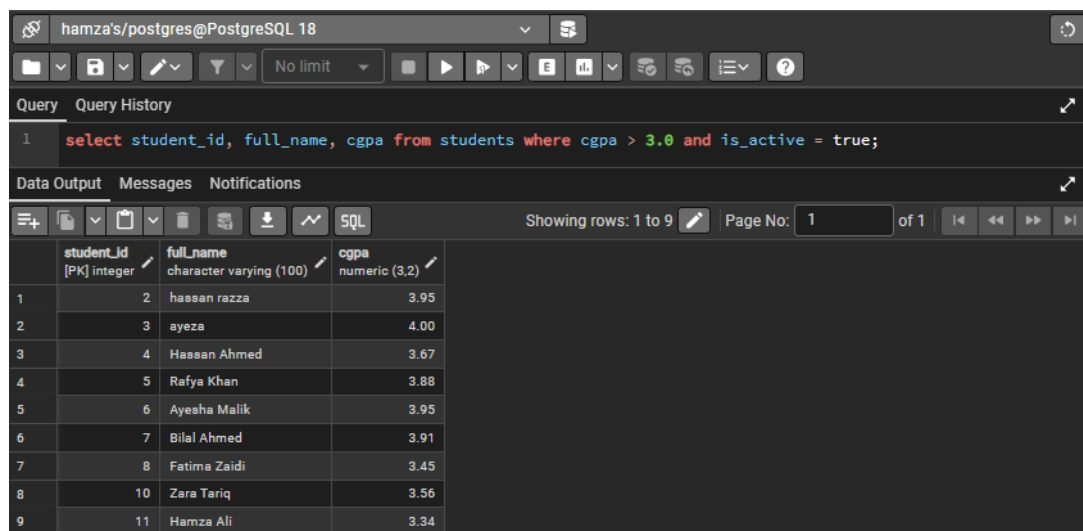
Figure 10: CGPA between 2.0 and 3.5



The screenshot shows a PostgreSQL query window with the following query: `select student_id, full_name, cgpa from students where full_name like 'A%';`. The results are displayed in a table with 1 row.

	student_id [PK] integer	full_name character varying (100)	cgpa numeric (3,2)
1	6	Ayesha Malik	3.95

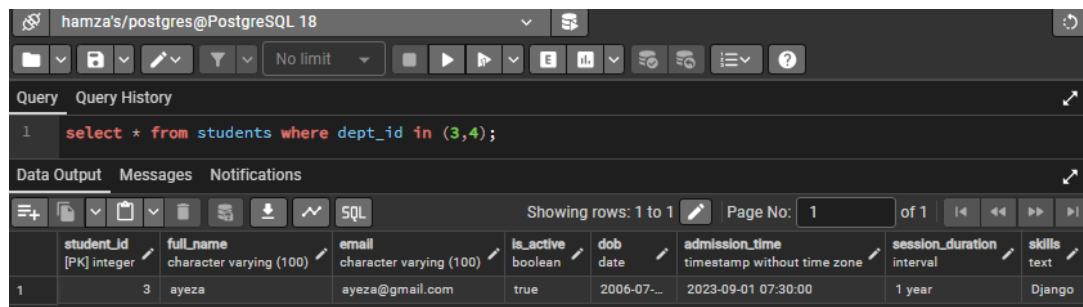
Figure 11: Name starting with A



The screenshot shows a PostgreSQL query window with the following query: `select student_id, full_name, cgpa from students where cgpa > 3.0 and is_active = true;`. The results are displayed in a table with 9 rows.

	student_id [PK] integer	full_name character varying (100)	cgpa numeric (3,2)
1	2	hassan razza	3.95
2	3	ayeza	4.00
3	4	Hassan Ahmed	3.67
4	5	Rafya Khan	3.88
5	6	Ayesha Malik	3.95
6	7	Bilal Ahmed	3.91
7	8	Fatima Zaidi	3.45
8	10	Zara Tariq	3.56
9	11	Hamza Ali	3.34

Figure 12: CGPA greater than 3.0 and Active



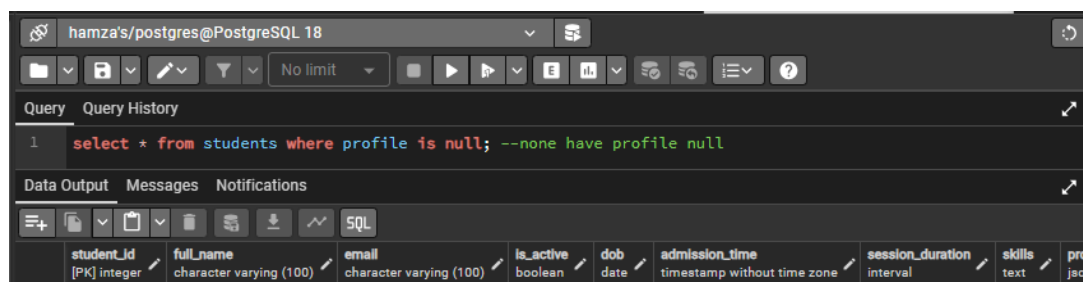
hamza's/postgres@PostgreSQL 18

Query: `select * from students where dept_id in (3,4);`

Data Output: Showing rows: 1 to 1 | Page No: 1 of 1

	student_id	full_name	email	is_active	dob	admission_time	session_duration	skills
	[PK] integer	character varying (100)	character varying (100)	boolean	date	timestamp without time zone	interval	text
1	3	ayeza	ayeza@gmail.com	true	2006-07-...	2023-09-01 07:30:00	1 year	Django

Figure 13: CS and SE departments



hamza's/postgres@PostgreSQL 18

Query: `select * from students where profile is null; --none have profile null`

Data Output: Showing rows: 1 to 1 | Page No: 1 of 1

	student_id	full_name	email	is_active	dob	admission_time	session_duration	skills	profile
	[PK] integer	character varying (100)	character varying (100)	boolean	date	timestamp without time zone	interval	text	json

Figure 14: Students with Null Profiles

3.2 Update and Delete Operations

```

1 UPDATE students
2 SET cgpa = LEAST (cgpa + 0.2 , 4.00) -- to keep gpa within set
   slimits
3 WHERE student_id IN (
4     SELECT student_id
5     FROM enrollments
6     GROUP BY student_id
7     HAVING AVG(attendance) > 85 --average was taken as multiple
   courses in each table
8 );
9
10 delete from students where cgpa <1.5;
11
12 delete from enrollments where attendance < 50; -- only deletes
   row by row
13
14 truncate table enrollments; --deletes everything, resets table

```



```

1 UPDATE students
2 SET cgpa = LEAST (cgpa + 0.2 , 4.00) -- to keep gpa within set slimits
3 WHERE student_id IN (
4     SELECT student_id
5     FROM enrollments
6     GROUP BY student_id
7     HAVING AVG(attendance) > 85 --average was taken as multiple courses in each table
8 );

```

UPDATE 4

Query returned successfully in 100 msec.

Figure 15: Adding 0.2 to CGPA if avg attendance ≥ 85

```

1 delete from students where cgpa <1.5;

```

DELETE 0

Query returned successfully in 82 msec.

Figure 16: Deleting CGPA ≤ 1.5

```

1 delete from enrollments where attendance < 50; -- only deletes row by row
2
3 truncate table enrollments; --deletes everything, resets table
4
5

```

TRUNCATE TABLE

Query returned successfully in 85 msec.

Figure 17: Difference between Delete and Truncate

Task 4 Aggregation and Analytical Queries

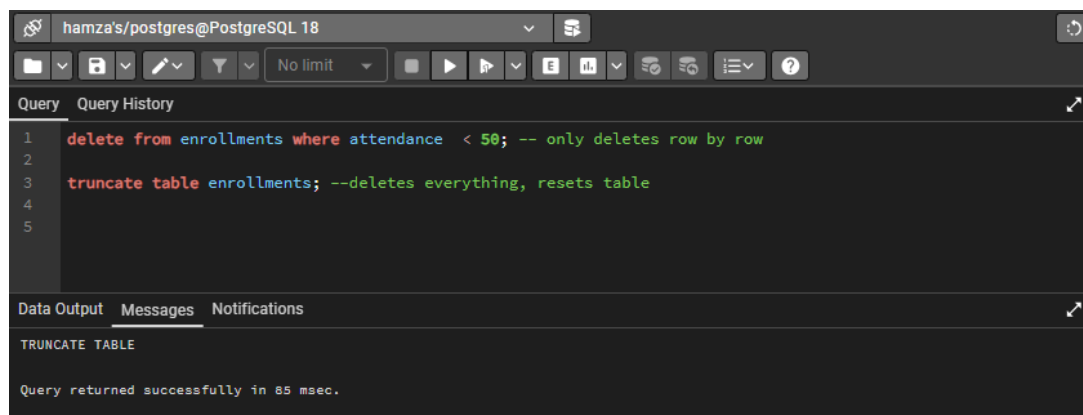
```

1.
2 select avg(s.cgpa), d.dept_name from students s join departments
   d
3 on s.dept_id = d.dept_id group by (dept_name); --joining is
   required when data in multiple tables
2.

```

```
5  select
6      s.student_id,
7      s.full_name,
8      count(e.course_id) as courses_enrolled
9  from students s
10 join enrollments e on s.student_id = e.student_id
11 group by s.student_id, s.full_name
12 having count(e.course_id) > 2;
13
14 3.
15 SELECT
16     s.student_id,
17     s.full_name,
18     COUNT(CASE WHEN e.marks < 50 THEN 1 END) as failed_courses
19 FROM students s
20 JOIN enrollments e ON s.student_id = e.student_id
21 GROUP BY s.student_id, s.full_name
22 HAVING COUNT(CASE WHEN e.marks < 50 THEN 1 END) > 1
23 ORDER BY failed_courses DESC;
24
25 4.
26 SELECT
27     s.student_id,
28     s.full_name,
29     ROUND(AVG(e.marks), 2) as avg_marks,
30     COUNT(e.course_id) as courses_taken
31 FROM students s
32 JOIN enrollments e ON s.student_id = e.student_id
33 GROUP BY s.student_id, s.full_name
34 ORDER BY avg_marks DESC
35 LIMIT 3;
36
37 5.
38 SELECT
39     semester,
40     COUNT(*) as total_enrollments,
41     COUNT(DISTINCT student_id) as unique_students,
42     ROUND(AVG(marks), 2) as avg_marks
43 FROM enrollments
44 GROUP BY semester
45 ORDER BY semester DESC;
46
47 6.
48 SELECT
49     DATE(admission_time) as admission_date,
50     COUNT(*) as students_admitted,
51     ROUND(AVG(cgpa), 2) as avg_cgpa
52 FROM students
53 GROUP BY DATE(admission_time);
54
55 7.
56 SELECT
57     d.dept_name,
58     ROUND(AVG(s.cgpa), 2) as avg_cgpa,
59     COUNT(s.student_id) as student_count,
60     MIN(s.cgpa) as min_cgpa,
```

```
56     MAX(s.cgpa) as max_cgpa
57 FROM students s
58 JOIN departments d ON s.dept_id = d.dept_id
59 GROUP BY d.dept_name
60 HAVING AVG(s.cgpa) > 3.0
61 ORDER BY avg_cgpa DESC;
62 8.
63 SELECT
64     s.student_id,
65     s.full_name,
66     SUM(e.marks) as total_marks,
67     AVG(e.marks) as avg_marks,
68     COUNT(e.course_id) as courses
69 FROM students s
70 JOIN enrollments e ON s.student_id = e.student_id
71 GROUP BY s.student_id, s.full_name
72 ORDER BY total_marks DESC;
```

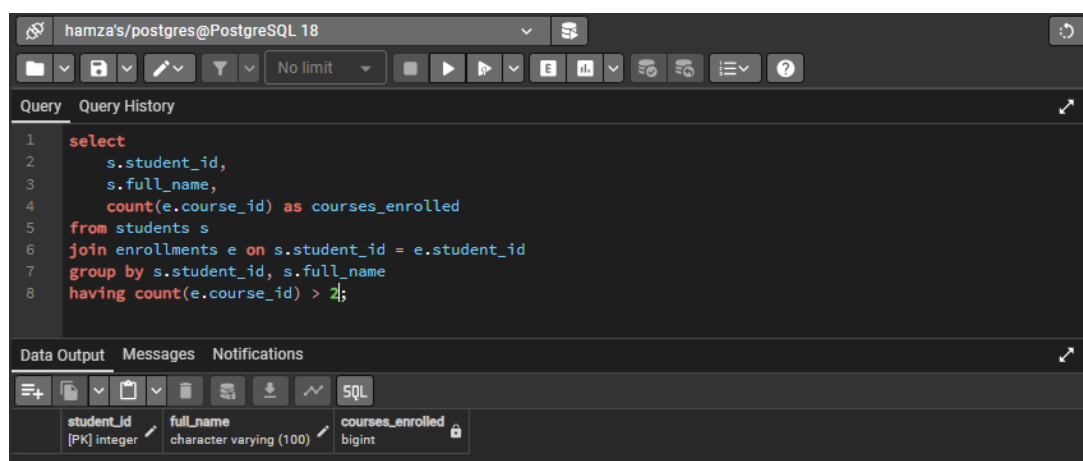


The screenshot shows a PostgreSQL 18 query editor window. The query editor contains the following SQL statements:

```
1 delete from enrollments where attendance < 50; -- only deletes row by row
2
3 truncate table enrollments; --deletes everything, resets table
4
5
```

The "Data Output" tab is selected, showing the message:

```
TRUNCATE TABLE
Query returned successfully in 85 msec.
```



The screenshot shows a PostgreSQL 18 query editor window. The query editor contains the following SQL statement:

```
1 select
2     s.student_id,
3     s.full_name,
4     count(e.course_id) as courses_enrolled
5 from students s
6 join enrollments e on s.student_id = e.student_id
7 group by s.student_id, s.full_name
8 having count(e.course_id) > 2;
```

The "Data Output" tab is selected, showing the message:

```
SQL
```

Below the message, the column definitions are displayed:

student_id	full_name	courses_enrolled
[PK] integer	character varying (100)	bigint

hamza's/postgres@PostgreSQL 18

Query Query History

```

2  SELECT
3      s.student_id,
4      s.full_name,
5      COUNT(CASE WHEN e.marks < 50 THEN 1 END) as failed_courses
6  FROM students s
7  JOIN enrollments e ON s.student_id = e.student_id
8  GROUP BY s.student_id, s.full_name
9  HAVING COUNT(CASE WHEN e.marks < 50 THEN 1 END) > 1
10 ORDER BY failed_courses DESC;

```

Data Output Messages Notifications

student_id	full_name	failed_courses
[PK] integer	character varying (100)	bigint

hamza's/postgres@PostgreSQL 18

Query Query History

```

1  SELECT
2      s.student_id,
3      s.full_name,
4      ROUND(AVG(e.marks), 2) as avg_marks,
5      COUNT(e.course_id) as courses_taken
6  FROM students s
7  JOIN enrollments e ON s.student_id = e.student_id
8  GROUP BY s.student_id, s.full_name
9  ORDER BY avg_marks DESC
10 LIMIT 3;

```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	student_id	full_name	avg_marks	courses_taken
1	3	ayeza	96.50	2
2	2	hassan razza	92.50	2
3	11	Hamza Ali	92.00	2

hamza's/postgres@PostgreSQL 18

Query Query History

```

1  SELECT
2      semester,
3      COUNT(*) as total_enrollments,
4      COUNT(DISTINCT student_id) as unique_students,
5      ROUND(AVG(marks), 2) as avg_marks
6  FROM enrollments
7  GROUP BY semester
8  ORDER BY semester DESC;

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	semester	total_enrollments	unique_students	avg_marks
1	Spring 20...	22	12	79.32

hamza's/postgres@PostgreSQL 18

Query Query History

```
1 SELECT
2     DATE(admission_time) as admission_date,
3     COUNT(*) as students_admitted,
4     ROUND(AVG(cgpa), 2) as avg_cgpa
5 FROM students
6 GROUP BY DATE(admission_time);
```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of 1

	admission_date date	students_admitted bigint	avg_cgpa numeric
1	2024-01-10	1	3.56
2	2022-09-01	4	3.14
3	2022-01-17	1	2.78
4	2023-01-15	1	3.91
5	2023-09-01	7	3.65
6	2024-09-01	1	2.89

hamza's/postgres@PostgreSQL 18

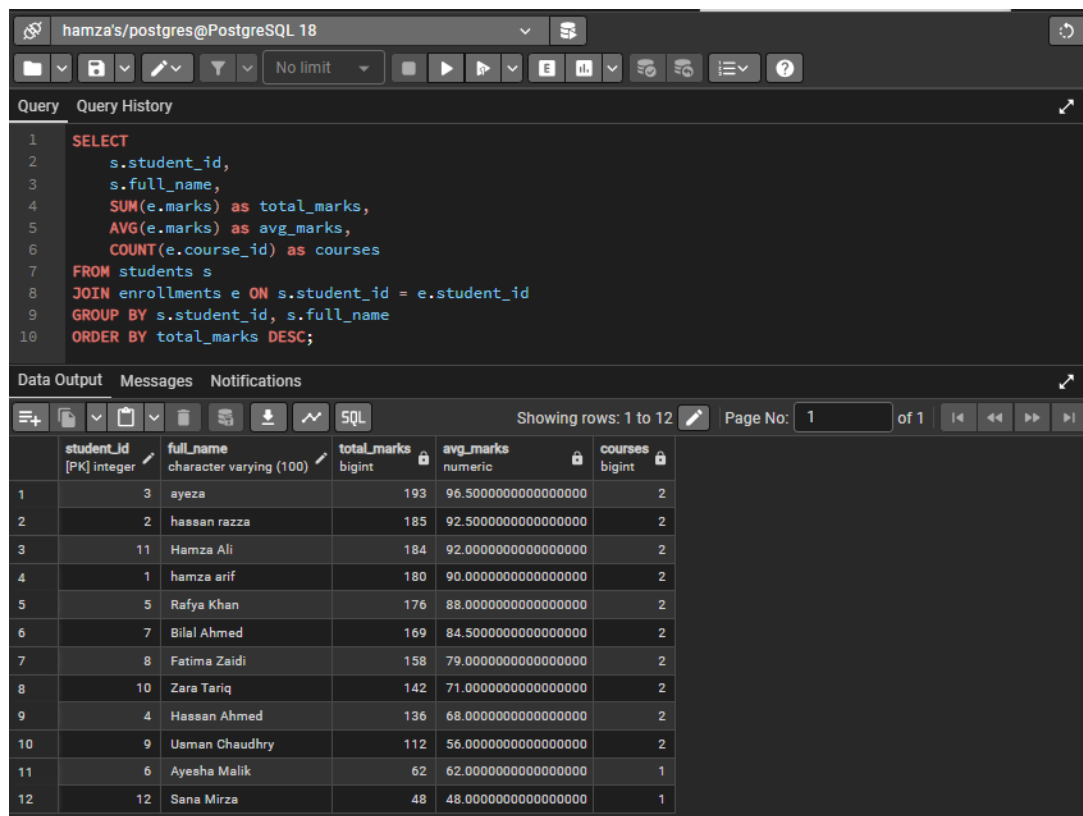
Query Query History

```
1 |
2 SELECT
3     d.dept_name,
4     ROUND(AVG(s.cgpa), 2) as avg_cgpa,
5     COUNT(s.student_id) as student_count,
6     MIN(s.cgpa) as min_cgpa,
7     MAX(s.cgpa) as max_cgpa
8 FROM students s
9 JOIN departments d ON s.dept_id = d.dept_id
10 GROUP BY d.dept_name
11 HAVING AVG(s.cgpa) > 3.0
12 ORDER BY avg_cgpa DESC;
```

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of 1

	admission_date date	students_admitted bigint	avg_cgpa numeric
1	2024-01-10	1	3.56
2	2022-09-01	4	3.14
3	2022-01-17	1	2.78
4	2023-01-15	1	3.91
5	2023-09-01	7	3.65
6	2024-09-01	1	2.89



The screenshot shows a PostgreSQL 18 query editor with the following SQL query:

```

1 SELECT
2   s.student_id,
3   s.full_name,
4   SUM(e.marks) as total_marks,
5   AVG(e.marks) as avg_marks,
6   COUNT(e.course_id) as courses
7 FROM students s
8 JOIN enrollments e ON s.student_id = e.student_id
9 GROUP BY s.student_id, s.full_name
10 ORDER BY total_marks DESC;

```

The query results are displayed in a table with the following columns: student_id, full_name, total_marks, avg_marks, and courses. The results are ordered by total_marks in descending order.

	student_id	full_name	total_marks	avg_marks	courses
1	3	ayeza	193	96.50000000000000	2
2		hassan razza	185	92.50000000000000	2
3	11	Hamza Ali	184	92.00000000000000	2
4	1	hamza arif	180	90.00000000000000	2
5	5	Rafya Khan	176	88.00000000000000	2
6	7	Bilal Ahmed	169	84.50000000000000	2
7	8	Fatima Zaidi	158	79.00000000000000	2
8	10	Zara Tariq	142	71.00000000000000	2
9	4	Hassan Ahmed	136	68.00000000000000	2
10	9	Usman Chaudhry	112	56.00000000000000	2
11	6	Ayesha Malik	62	62.00000000000000	1
12	12	Sana Mirza	48	48.00000000000000	1

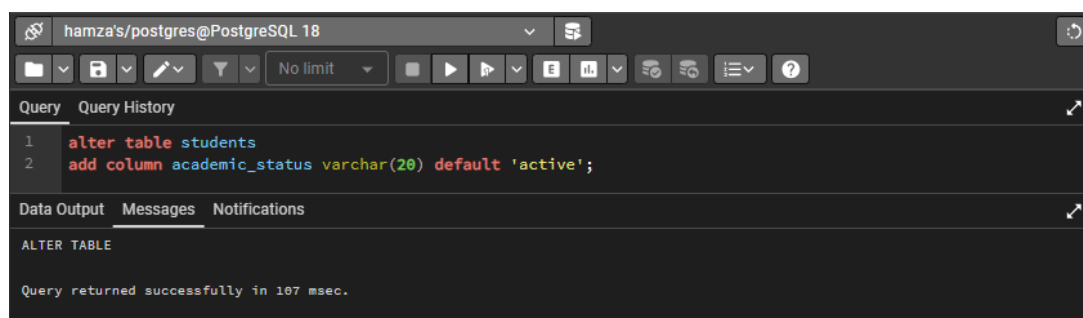
Task 5 Academic Probation and Integrity Enforcement

5.1 Part A Schema Modification

```

1
2 alter table students
3 add column academic_status varchar(20) default 'active';

```



The screenshot shows the same PostgreSQL 18 query editor with the following SQL query:

```

1 alter table students
2 add column academic_status varchar(20) default 'active';

```

The query results are displayed in a table with the following columns: student_id, full_name, total_marks, avg_marks, and courses. The results are ordered by total_marks in descending order.

	student_id	full_name	total_marks	avg_marks	courses
1	3	ayeza	193	96.50000000000000	2
2		hassan razza	185	92.50000000000000	2
3	11	Hamza Ali	184	92.00000000000000	2
4	1	hamza arif	180	90.00000000000000	2
5	5	Rafya Khan	176	88.00000000000000	2
6	7	Bilal Ahmed	169	84.50000000000000	2
7	8	Fatima Zaidi	158	79.00000000000000	2
8	10	Zara Tariq	142	71.00000000000000	2
9	4	Hassan Ahmed	136	68.00000000000000	2
10	9	Usman Chaudhry	112	56.00000000000000	2
11	6	Ayesha Malik	62	62.00000000000000	1
12	12	Sana Mirza	48	48.00000000000000	1

5.2 Part B Analytical Computation

```

1 select full_name, s.student_id, count(course_id)
2   from students s join enrollments e on e.student_id = s.
3   student_id
4   group by(s.student_id, full_name);
5 select

```

```

6      s.full_name,s.student_id,count(*) as failed_courses_count
7      from students s
8      join enrollments e on e.student_id = s.student_id
9      where e.marks < 50
10     group by s.student_id, s.full_name;
11
12 select
13     s.full_name,s.student_id,avg(e.marks)
14     from students s
15     join enrollments e on e.student_id = s.student_id
16     group by s.student_id, s.full_name;

```

hamza's/postgres@PostgreSQL 18

Query Query History

```

1 select full_name , s . student_id , count ( course_id )
2 from students s join enrollments e on e . student_id = s .
3 student_id group by ( s . student_id , full_name ) ;

```

Data Output Messages Notifications

Showing rows: 1 to 12 Page No: 1 of 1

	full_name character varying (100)	student_id [PK] integer	count bigint
1	Hamza Ali	11	2
2	Usman Chaudhry	9	2
3	ayeza	3	2
4	Rafya Khan	5	2
5	Hassan Ahmed	4	2
6	Zara Tariq	10	2
7	Ayesha Malik	6	1
8	haasan razza	2	2
9	Bilal Ahmed	7	2
10	Sana Mirza	12	1
11	hamza arif	1	2
12	Fatima Zaidi	8	2

hamza's/postgres@PostgreSQL 18

Query Query History

```

1 select
2     s.full_name,
3     s.student_id,
4     count(*) as failed_courses_count
5 from students s
6 join enrollments e on e.student_id = s.student_id
7 where e.marks < 50
8 group by s.student_id, s.full_name;

```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	full_name character varying (100)	student_id [PK] integer	failed_courses_count bigint
1	Sana Mirza	12	1

The screenshot shows a PostgreSQL query editor with the following query:

```

1 select
2   s.full_name,
3   s.student_id,
4   avg(e.marks)
5 from students s
6 join enrollments e on e.student_id = s.student_id
7 group by s.student_id, s.full_name;

```

The results are displayed in a table with the following columns: full_name, student_id, and avg. The table contains 12 rows of data.

	full_name	student_id	avg
1	Hamza Ali	11	92.00000000000000
2	Usman Chaudhry	9	56.00000000000000
3	ayeza	3	96.50000000000000
4	Rafya Khan	5	88.00000000000000
5	Hassan Ahmed	4	68.00000000000000
6	Zara Tariq	10	71.00000000000000
7	Ayesha Malik	6	62.00000000000000
8	hassan razza	2	92.50000000000000
9	Bilal Ahmed	7	84.50000000000000
10	Sana Mirza	12	48.00000000000000
11	hamza arif	1	90.00000000000000
12	Fatima Zaidi	8	79.00000000000000

5.3 Part C Conditional Status Update

- ```

1 update students set academic_status = 'probation'
2 where student_id in (select s.student_id from students s
3 left join enrollments e on s.student_id = e.student_id
4 group by s.student_id, s.cgpa having s.cgpa < 2.0 or
5 sum(case when e.marks < 50 then 1 else 0 end) > 2);

```
- ```

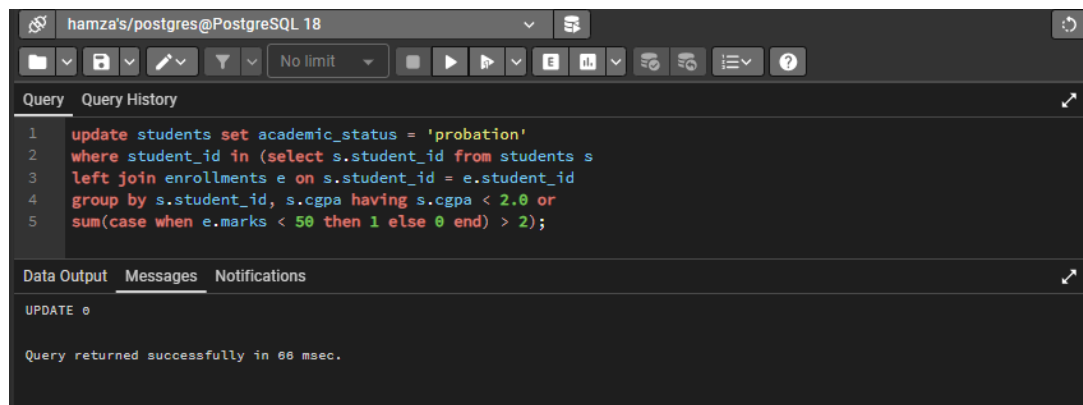
8 update students set academic_status = 'excellent'
9 where student_id in (select s.student_id from students s
10 left join enrollments e on s.student_id = e.student_id
11 group by s.student_id, s.cgpa having s.cgpa > 2.0 and
12 avg(e.marks) > 85);

```
- ```

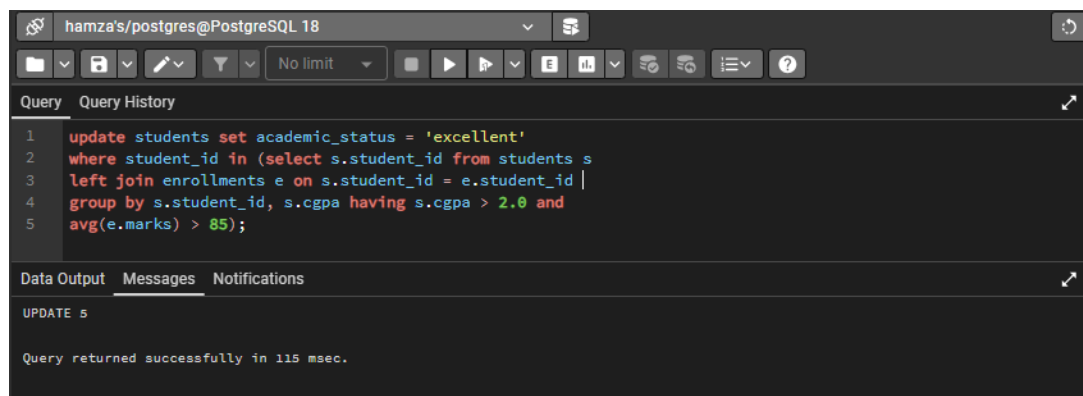
14 update students set academic_status = 'inactive'
15 where student_id not in (select distinct student_id from
 enrollments);

```

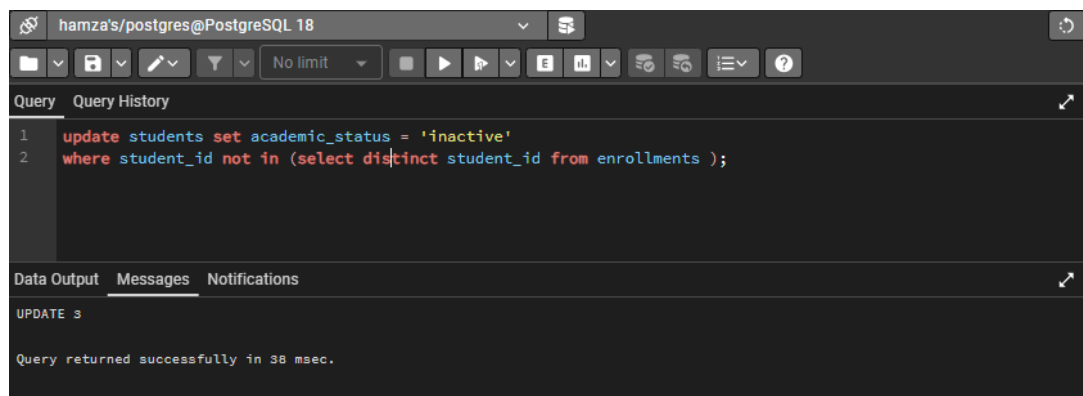




```
hamza's/postgres@PostgreSQL 18
Query Query History
1 update students set academic_status = 'probation'
2 where student_id in (select s.student_id from students s
3 left join enrollments e on s.student_id = e.student_id
4 group by s.student_id, s.cgpa having s.cgpa < 2.0 or
5 sum(case when e.marks < 50 then 1 else 0 end) > 2);
Data Output Messages Notifications
UPDATE 0
Query returned successfully in 66 msec.
```



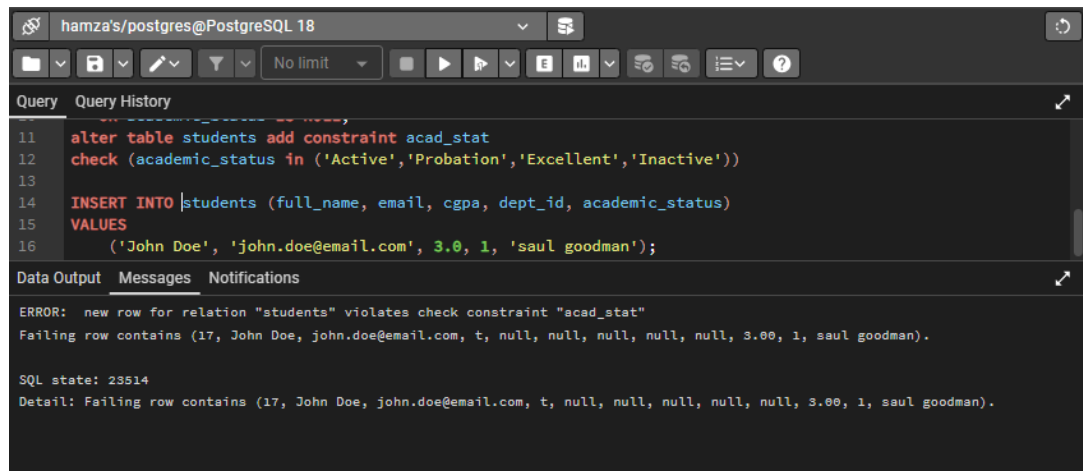
```
hamza's/postgres@PostgreSQL 18
Query Query History
1 update students set academic_status = 'excellent'
2 where student_id in (select s.student_id from students s
3 left join enrollments e on s.student_id = e.student_id |
4 group by s.student_id, s.cgpa having s.cgpa > 2.0 and
5 avg(e.marks) > 85);
Data Output Messages Notifications
UPDATE 5
Query returned successfully in 115 msec.
```



```
hamza's/postgres@PostgreSQL 18
Query Query History
1 update students set academic_status = 'inactive'
2 where student_id not in (select distinct student_id from enrollments);
Data Output Messages Notifications
UPDATE 3
Query returned successfully in 38 msec.
```

## 5.4 Part D Constraint Reinforcement

```
1 alter table students add constraint acad_stat
2 check (academic_status in ('Active','Probation','Excellent','
 Inactive'))
3
4 INSERT INTO students (full_name, email, cgpa, dept_id,
 academic_status)
5 VALUES
6 ('John_Doe', 'john.doe@email.com', 3.0, 1, 'saul_goodman');
```



The screenshot shows a PostgreSQL client window titled "hamza's/postgres@PostgreSQL 18". The interface includes a toolbar with icons for file operations, query execution, and help. Below the toolbar, there are tabs for "Query" and "Query History". The "Query" tab is active, displaying a SQL script with line numbers 11 through 16. The script consists of an `ALTER TABLE` statement to add a check constraint named `acad_stat` to the `students` table, followed by an `INSERT INTO` statement. The `INSERT` statement attempts to add a new row with the values `('John Doe', 'john.doe@email.com', 3.0, 1, 'saul goodman')`. Below the query editor, there are tabs for "Data Output", "Messages", and "Notifications". The "Messages" tab is active, showing an error message: "ERROR: new row for relation \"students\" violates check constraint \"acad\_stat\"". Below the error message, it states "Failing row contains (17, John Doe, john.doe@email.com, t, null, null, null, null, null, 3.00, 1, saul goodman)." and "SQL state: 23514". A detailed message below that says "Detail: Failing row contains (17, John Doe, john.doe@email.com, t, null, null, null, null, null, 3.00, 1, saul goodman)." The interface has a dark theme.

```
11 alter table students add constraint acad_stat
12 check (academic_status in ('Active','Probation','Excellent','Inactive'))
13
14 INSERT INTO students (full_name, email, cgpa, dept_id, academic_status)
15 VALUES
16 ('John Doe', 'john.doe@email.com', 3.0, 1, 'saul goodman');
```

ERROR: new row for relation "students" violates check constraint "acad\_stat"  
Failing row contains (17, John Doe, john.doe@email.com, t, null, null, null, null, null, 3.00, 1, saul goodman).

SQL state: 23514  
Detail: Failing row contains (17, John Doe, john.doe@email.com, t, null, null, null, null, null, 3.00, 1, saul goodman).